

Orchestrator: A Deterministic Execution Engine for Device Interaction Runs

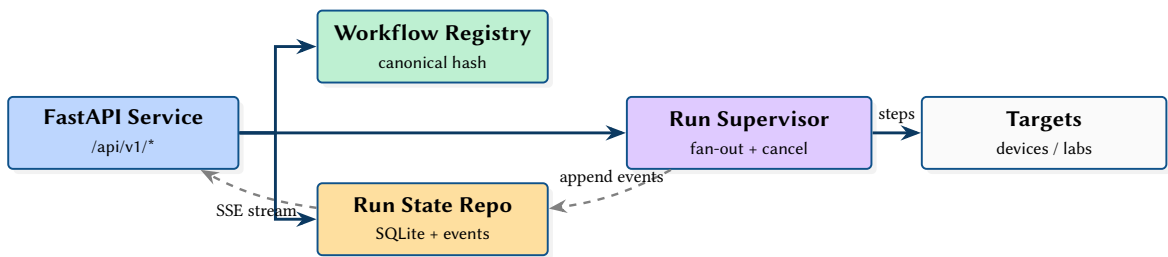
Run Coordination, Durable State, and Streaming Observability for Network Automation

Simon Knight

Network Automation Ecosystem

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. This report documents the Orchestrator service: an execution engine for reliable, replayable device interaction runs. The system offers a small workflow DSL (v1), schema-validated registration, per-run durability using SQLite, bounded concurrency, cancellation, and a server-sent event (SSE) stream of structured run events. The goal is deterministic run coordination with clean integration boundaries. We describe the design, core data models, APIs, and execution semantics, and present a set of diagrams that capture the system’s state machine, concurrency controls, and event flow. Finally, we detail the Orchestrator’s transition into a distributed control system, introducing advanced capabilities such as stochastic DAG evaluation (Monte Carlo testing), graph-theoretic concurrency, algebraic state reversion, formal invariant verification, decentralized edge federation via CRDTs, dynamic topological sandboxing, and the introduction of formal state attestation and shadow-fabric pre-commit for absolute operational certainty.

Contents

1	Problem Statement and Scope	2
2	Prior Art and the Limits of General-Purpose Runners	2
3	System Overview	2
4	Workflow Definition and Validation	2
5	Secrets Policy	3
6	Run Model, Events, and Durability	3
7	State Machine and Formalisms	4
7.1	Determinism as a Partial Order Over Events	4
7.2	Failure Taxonomy as a Policy Lattice	5
8	Execution Semantics	5
8.1	Concurrency as a Budgeted Scheduling Problem	6
8.2	Cancellation Propagation and Cooperative Checkpoints	6
9	Event Streaming (SSE)	7
9.1	Event Streams as a Projection of an Append-only Log	7
10	Artifacts and Information-Theoretic Redaction	7
11	Safety and Command Validation	8
12	Global Toolchain Orchestration	8
13	Advanced Capabilities & Ecosystem Insights	9
13.1	Goal-Backward Execution (Intent Reconciliation).	9
13.2	Causal “Time-Travel” Debugging	9
13.3	Formal Invariant Verification	9
13.4	Predictive Blast Radius Simulation	9
14	Formal State Attestation and Cryptographic Compliance	9
15	Shadow-Fabric Pre-Commit (Ephemeral Digital Twins)	10
16	Implementation Walkthrough (Repository Mapping)	10
17	v2.0 Architecture: Stochastic Workflows and Distributed Control	11
17.1	Stochastic DAG Evaluation (Monte Carlo Stress Testing)	11
17.2	Graph-Theoretic Concurrency and Blast-Radius Containment	11
17.3	Algebraic State Reversion (Automated Compensating Transactions)	11
17.4	Semantic Convergence Bounding	11
17.5	Decentralized Edge Federation via CRDTs	11
17.6	Dynamic Topological Sandboxing (Execution-Driven Traffic Engineering)	12
17.7	Symbolic Execution and Formal Verification (SMT Theorem Proving)	12
	Revision History	14
SSE	Server-Sent Events	7

1 Problem Statement and Scope

Network automation in real labs and testbeds requires more than a library of device operations. The missing component is a reliable coordinator that executes the same workflow across a target set with deterministic semantics, strong observability, and durable run history.

This repository implements the Orchestrator engine as an API-first service (FastAPI) and a small SDK for authoring and parsing workflow definitions. It is intentionally not a full Tower/AWX clone. The v1 scope is device-focused execution semantics and a control plane that can be embedded in CLI/Workbench/CI.

The authoritative requirements and sequencing are captured in the in-repo planning documents (`.planning/PROJECT.md`, `.planning/REQUIREMENTS.md`, `.planning/ROADMAP.md`) [1–3].

Design Decision 1: API-first Service

The orchestrator exposes a stable HTTP API as its primary interface. This decouples execution semantics from any single client UX and supports embedding into Workbench, CLI, and CI pipelines.

Design Decision 2: Device Interaction Boundary

The orchestrator owns: run coordination, state persistence, concurrency controls, retries/timeouts, cancellation, and event streaming. Transport, parsing, and device capability logic are delegated to a device interaction library via adapters.

2 Prior Art and the Limits of General-Purpose Runners

While general-purpose orchestrators (e.g., Airflow, Temporal, StackStorm) excel at IT automation and API choreography, they often fail catastrophically when applied to physical network infrastructure. These systems implicitly assume that operations are idempotent API calls and that state is reliably stored in a centralized database.

However, networks are complex, distributed graphs where operations are rarely strictly idempotent, and state is distributed physically across the topology. General-purpose runners lack semantic topological awareness—they will happily execute concurrent reboots on redundant spine switches if instructed to, inadvertently partitioning the network. Furthermore, they lack an understanding of protocol convergence, relying on brittle `sleep()` statements rather than waiting for true BGP or OSPF state synchronization. The Orchestrator is designed specifically to address these domain-specific mathematical and topological constraints.

3 System Overview

At a high level, Orchestrator consists of: (i) a workflow registry, (ii) a run state repository with event log, (iii) a supervisor that executes runs across targets, and (iv) an API layer that registers workflows, starts runs, cancels runs, and streams events.

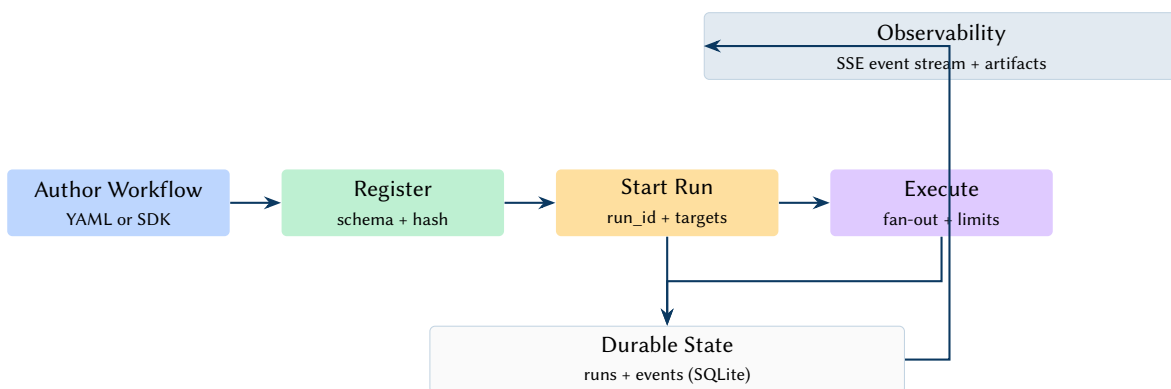


Figure 1. End-to-end lifecycle: authoring, registration, execution, and observability.

4 Workflow Definition and Validation

Workflows are represented as validated Pydantic models (namespace, name, DSL version, and an ordered list of steps). Each step declares an adapter, a typed inputs map, and explicit dependencies (`depends_on`).

Insight:
A run is not “a script”; it is a durable, replayable execution record. The core output of the engine is a consistent event stream that correlates run, target, and step.

The YAML parser is designed to avoid YAML 1.1 coercion pitfalls by enforcing YAML 1.2 “safe” parsing (ruamel.yaml, safe typ, explicit version). This is a pragmatic correctness guardrail.

Listing 1 A concrete example of the Workflow YAML DSL.

```
namespace: netops
name: deploy_acls
version: "1.0"
steps:
- id: backup_config
  adapter: deviceinteraction
  inputs:
    command: "show running-config"
- id: apply_acl
  depends_on: [backup_config]
  adapter: deviceinteraction
  inputs:
    config: "access-list 101 permit ip any any"
    credentials: secret://vault/network/routers
```

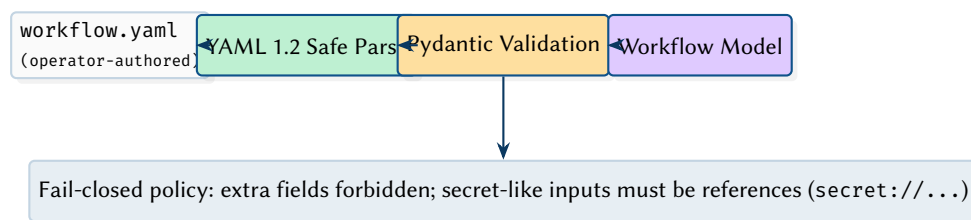


Figure 2. Workflow ingestion pipeline with validation and safety policy checks.

5 Secrets Policy

Step inputs are permitted to contain secret values only by reference. Sensitive keys (e.g., password, token, api_key) are rejected unless their values are a secret://... reference (either as a string or as a single-key map {secret_ref: ...}). This enforces “references only” as a structural invariant in the workflow model.

: Secret-by-Reference

Any step input whose key is sensitive MUST be expressed as a reference (secret://...); plaintext values are rejected at validation time.

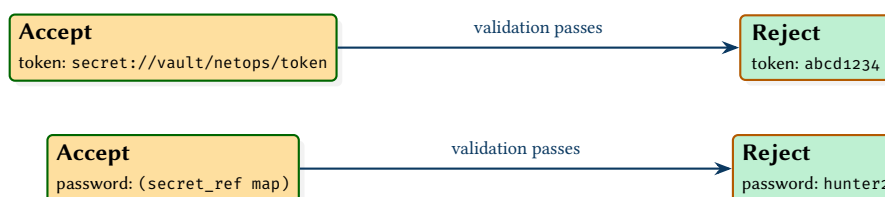


Figure 3. Secret input policy expressed as a validation invariant.

6 Run Model, Events, and Durability

Runs are persisted in SQLite with an append-only event log. A run references: the workflow identifier, the list of targets, and a run-level status. Each emitted event includes stable correlation identifiers (run, target, step) and a timestamp.

Listing 2 The anatomy of an emitted run event, capturing durable correlation identifiers.

```
{
  "event_id": "evt_5f8b9281a134...",
  "timestamp": "2026-03-18T10:45:00Z",
  "run_id": "run_9a2b7f32...",
  "target_id": "spine-01",
  "step_id": "apply_acl",
  "type": "StepCompleted",
  "payload": {
    "attempt": 1,
    "status": "success",
    "artifact_hash": "sha256:8f2a99bc..."
  }
}
```

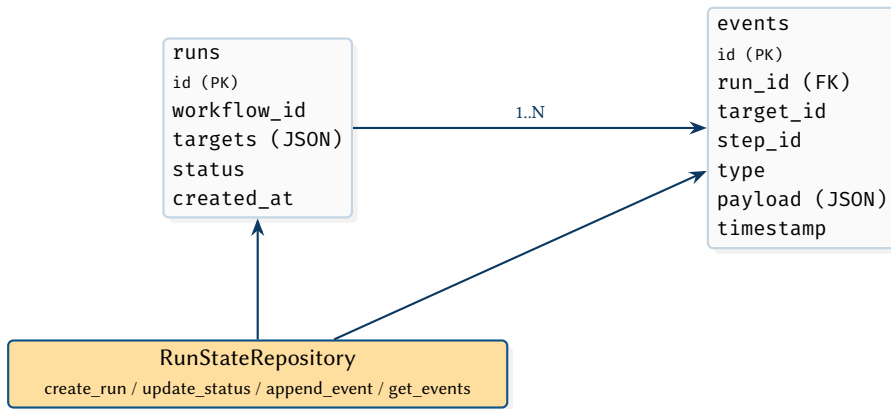


Figure 4. SQLite durability model: run header row plus correlated append-only events.

Design Decision 3: Event Log as Primary Interface

The event stream is treated as a first-class output: it enables live streaming to clients (SSE) and provides a durable audit trail for replay, debugging, and artifact correlation.

7 State Machine and Formalisms

Before detailing the mechanics of execution, we define the foundational mathematics that govern run lifecycle, ordering constraints, and failure handling.

The current implementation uses a run-level status set with terminal states COMPLETED, FAILED, and CANCELED.

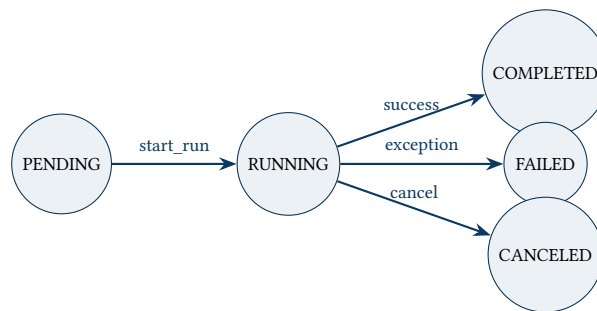


Figure 5. Run status state machine (v0.1.0).

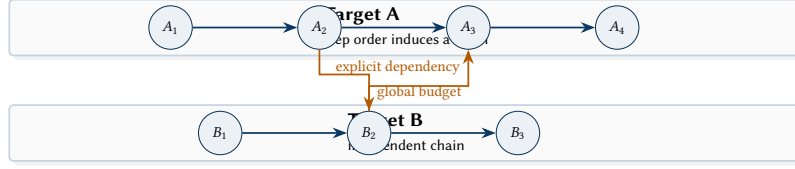
7.1 Determinism as a Partial Order Over Events

In distributed execution, “determinism” is not total ordering of all actions; it is the presence of explicit ordering constraints (a partial order) that define what must happen-before what. The orchestrator should make these constraints explicit in its event model and execution semantics.

Definition (Event set). Let E be the set of emitted run events. Each event $e \in E$ is identified by the tuple $(run_id, target_id, step_id, event_id)$ and carries a timestamp.

Definition (Happens-before). Define a relation $< \subseteq E \times E$ where $e_1 < e_2$ iff the orchestrator semantics require e_1 to occur before e_2 . This includes at least: (i) *Per-target step order* edges (for a fixed target, “Step Completed” for step k precedes “Step Started” for step $k+1$), and (ii) *Dependency* edges induced by `depends_on`.

Definition (Determinism boundary). Two executions are equivalent for clients if they produce event streams that are linear extensions of the same partial order $(E, <)$. This allows concurrency-induced interleavings, while forbidding reorderings that violate required constraints.



A partial order $<$ defines determinism boundaries: consumers may see interleavings that preserve edges, but not a total ordering across targets.

Figure 6. Determinism as a partial order: per-target step chains plus explicit cross-target constraints (budgets/dependencies) define valid interleavings.

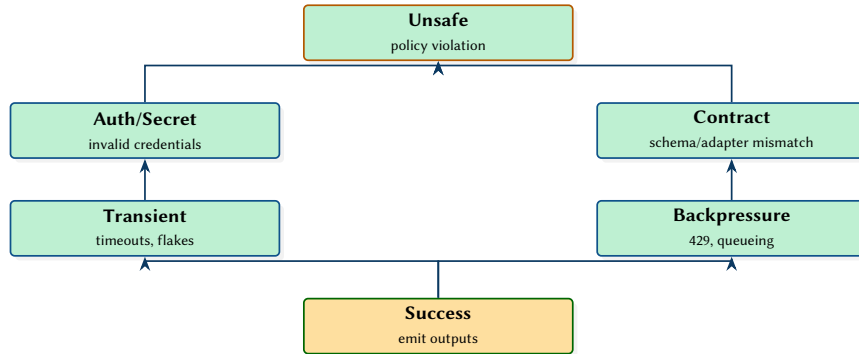
7.2 Failure Taxonomy as a Policy Lattice

Operational failures are not uniform. A robust orchestrator needs a taxonomy that maps observed errors to policy decisions (retry, skip, fail-fast, cancel, or quarantine). This can be expressed as a lattice where increasing “severity” restricts allowable actions.

Definition (Error class). Let C be a finite set of error classes (e.g., Transient, Backpressure, Auth/Secret, Contract, Unsafe). Each step attempt that fails is assigned a class $c \in C$.

Definition (Policy actions). Let A be the set of admissible actions: $A = \{\text{RETRY}, \text{DELAY}, \text{SKIP}, \text{FAILFAST}, \text{CANCEL}, \text{QUARANTINE}\}$. The policy returns a subset $\pi(c) \subseteq A$.

Definition (Severity order). Define a preorder $\sqsubseteq$ over classes such that $c_1 \sqsubseteq c_2$ means “ c_2 is at least as severe as c_1 ”. The policy is monotone when $c_1 \sqsubseteq c_2 \Rightarrow \pi(c_2) \subseteq \pi(c_1)$, i.e., more severe errors allow fewer actions.



Insight:
The lattice framing prevents accidental policy drift: adding a new “more severe” class cannot expand the action set without explicitly violating monotonicity.

Policy mapping (illustrative):

Transient $\rightarrow$ retry with backoff; Backpressure $\rightarrow$ delay + jitter; Auth/Secret $\rightarrow$ fail-fast (no retry); Contract $\rightarrow$ fail-fast (developer action); Unsafe $\rightarrow$ cancel/quarantine and emit audit event.

Figure 7. Failure taxonomy lattice: severity restricts admissible actions; policies map error classes to decisions while preserving auditability.

8 Execution Semantics

The supervisor executes the run by fanning out over targets and iterating the workflow steps per target. Cancellation is implemented via AnyIO cancel scopes. Concurrency is bounded by a global semaphore and optional per-target semaphores.

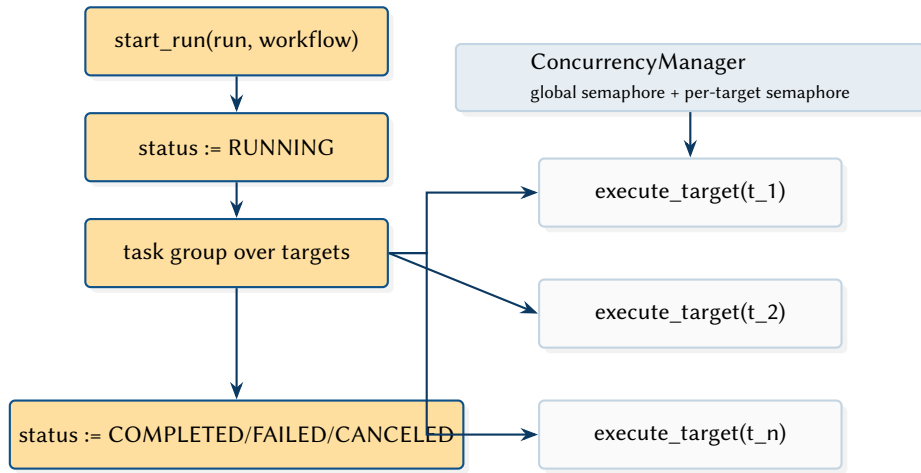


Figure 8. Run execution: fan-out across targets, step iteration per target, bounded by concurrency controls.

8.1 Concurrency as a Budgeted Scheduling Problem

Linear architecture diagrams are useful as orientation, but the core of the orchestrator is a set of execution invariants: global and per-target capacity constraints, and a scheduler that admits work only when the relevant budgets allow it. This can be expressed as a token budget model.

Insight:
Determinism is primarily a control-plane property: bounded concurrency and explicit ordering rules reduce “action at a distance” between targets during execution.

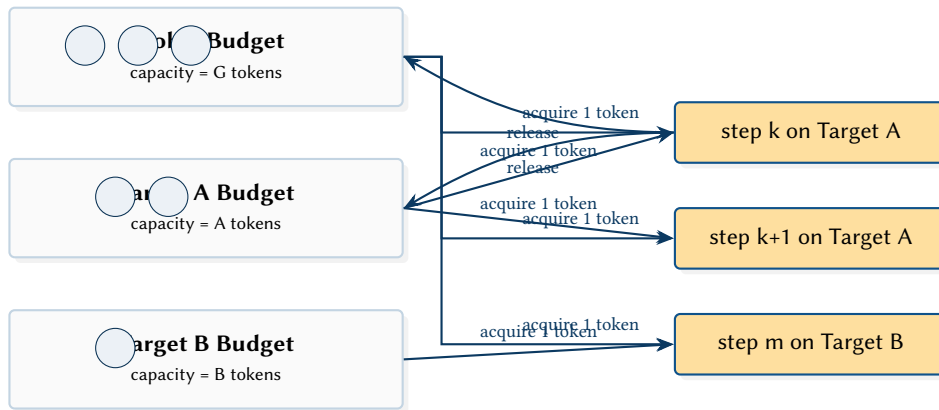


Figure 9. Concurrency as token budgets: each step admission consumes one global token and (optionally) one target token, bounding total parallelism and per-target pressure.

8.2 Cancellation Propagation and Cooperative Checkpoints

Cancellation is not simply a boolean state; it is a propagation mechanism. In AnyIO, a cancel scope is a tree: cancellation flows downward to child tasks at well-defined checkpoints (await points), making cancellation latency a function of where tasks yield.

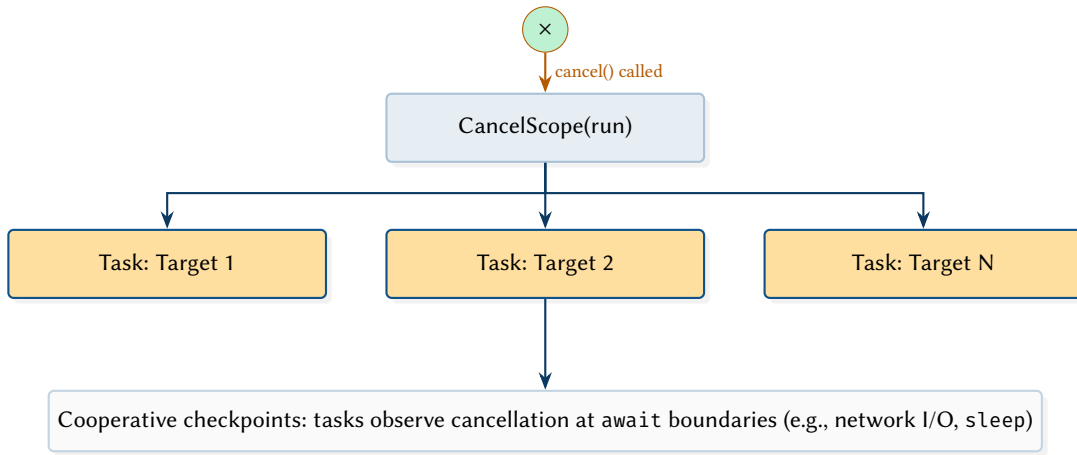


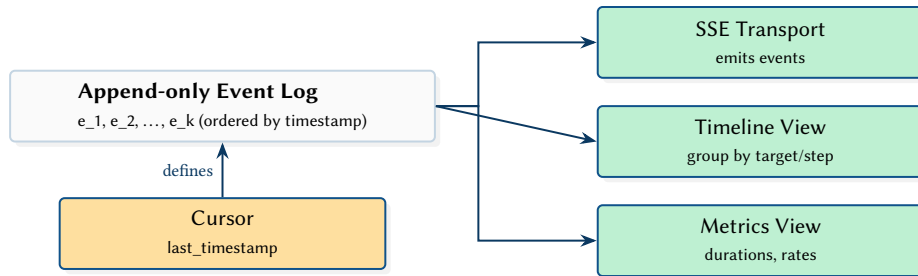
Figure 10. Cancellation propagation as a scope tree: a run-level cancel scope cancels all target tasks; responsiveness depends on cooperative await points.

9 Event Streaming (SSE)

Clients can subscribe to an SSE endpoint to receive a live stream of events. The event generator polls for new events after a cursor timestamp, yields them as SSE messages, and terminates once the run reaches a terminal state. This design uses a standard server-to-client streaming mechanism over HTTP (Server-Sent Events (SSE)) [4].

9.1 Event Streams as a Projection of an Append-only Log

The SSE endpoint is best understood as a projection: a client-specific view over a durable, append-only event log. This framing matters because it separates (i) the source of truth (events in SQLite) from (ii) transport (SSE) and (iii) derived views (UI timelines, metrics, summaries).



Invariant: events are append-only; consumers compute projections without mutating the source log.

Figure 11. Event streaming as projection: durable append-only events support multiple derived views; SSE is one transport for a cursor-based projection.

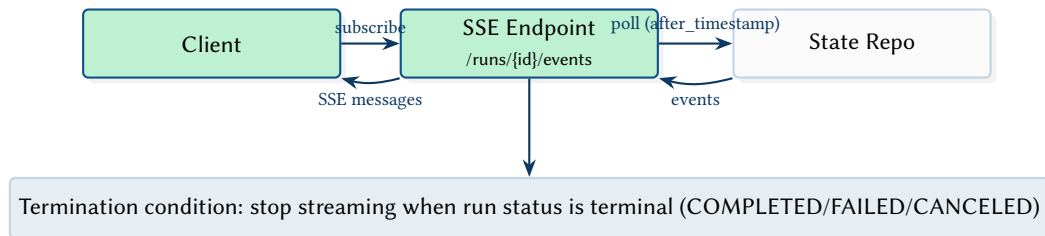


Figure 12. SSE event streaming architecture: API polls the durable event log and emits SSE messages to clients.

10 Artifacts and Information-Theoretic Redaction

The Orchestrator captures the output of device interactions as structured artifacts. Because device configurations and operational states often contain sensitive data (e.g., passwords, SNMP strings, cryptographic keys), the engine implements a mathematically bounded redaction pipeline before artifacts are persisted.

Rather than relying solely on brittle regular expressions, the Orchestrator achieves *Bounded Data Leakage* through zero-knowledge principles. Because of the “Secret-by-Reference” policy defined earlier, the Orchestrator never holds the plaintext secret in its own configuration state. The Redactor component intercepts the raw output stream from the Target Adapter. Since the orchestrator delegates secret decryption exclusively to the edge adapter or a Just-In-Time (JIT) broker, it processes the artifact stream with zero-knowledge of the root credentials, fundamentally limiting the blast radius of any potential data leak.

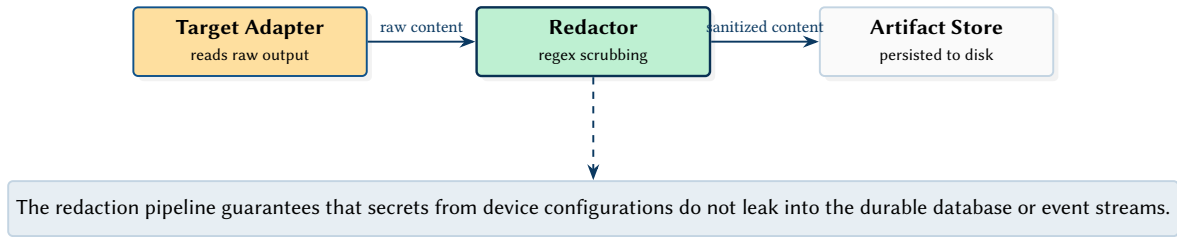


Figure 13. The automated artifact redaction pipeline.

11 Safety and Command Validation

To prevent catastrophic operational mistakes, the execution engine routes all target commands through a CommandValidator. This component acts as a gateway, evaluating commands against explicit allow and deny lists before they are dispatched to the network.

Additionally, the Orchestrator supports a simulation mode (e.g., `dry_run` flag or the `net sim` adapter). In this mode, the supervisor executes the full workflow logic but bypasses physical device mutation, allowing operators to validate intent safely.

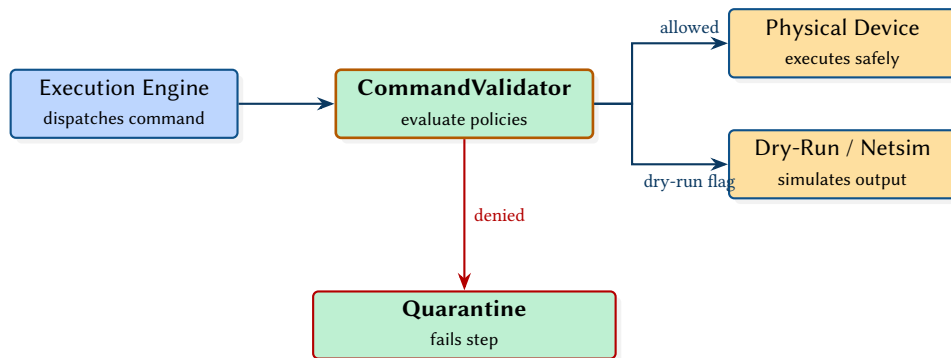


Figure 14. Command validation and routing logic ensuring operational safety.

12 Global Toolchain Orchestration

The Orchestrator transcends simple script execution by serving as the unified backend for the broader ecosystem’s ‘Define → Simulate → Visualize’ pipeline. By extracting toolchain orchestration into a common backend, both the `orchestrator-cli` and the graphical Workbench provide consistent behavior.

The toolchain flow ingests operator intent (definition), passes it through simulated environments (`net sim`) to calculate state transitions, and streams the results back for visualization or further physical deployment.

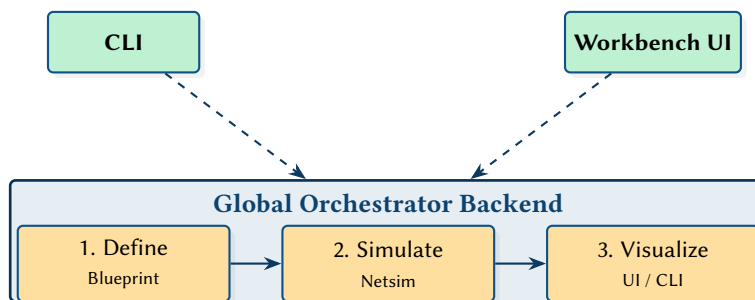


Figure 15. The Orchestrator unifying the ecosystem toolchain across diverse frontends.

13 Advanced Capabilities & Ecosystem Insights

The Orchestrator’s foundational models—immutable event logs, strict partial ordering, and policy-driven validation—enable capabilities that extend beyond traditional task execution. By treating execution as a mathematical control system rather than a script runner, the engine supports advanced features integral to the ‘Whiteboard-to-Plan’ (W2P) strategy. These concepts draw heavily on established principles of intent-based networking (IBN) [5] and distributed systems theory.

13.1 Goal-Backward Execution (Intent Reconciliation)

Instead of imperatively defining *what* to do, the user provides a final W2P Blueprint defining the desired state. The Orchestrator queries the live network to construct the current operational state, calculates the algebraic delta between this state and the intent, and dynamically synthesizes the execution graph required to reconcile the difference.

Automation thus becomes a process of calculated state transitions rather than linear scripting. This shift from imperative instructions to declarative state reconciliation is the hallmark of advanced control systems, ensuring idempotency and reducing configuration drift.

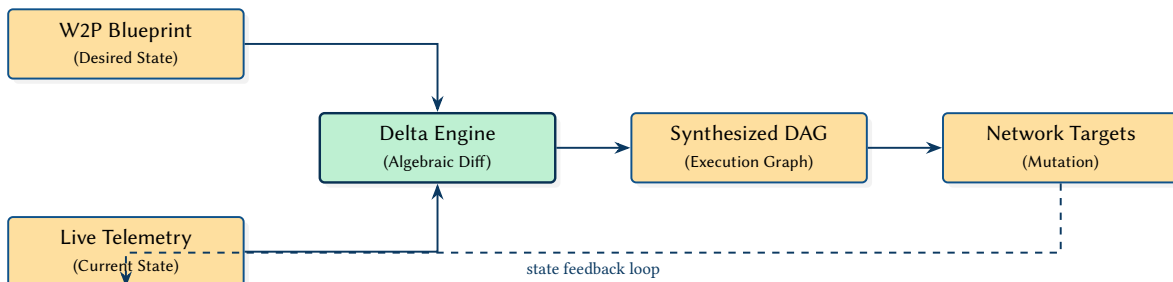


Figure 16. Goal-backward execution: The Orchestrator calculates the delta between intent and reality to synthesize a mitigating execution graph.

13.2 Causal “Time-Travel” Debugging

Because concurrency constraints are modeled as a strict partial order over an append-only SQLite event log, the Orchestrator provides a foundation for deterministic replay. Drawing on Lamport’s foundational work on logical clocks and event ordering [6], the event log accurately captures the happen-before relationships of all distributed operations.

When a complex deployment fails, the engine can mathematically replay the event stream backward. It isolates the exact concurrency interleaving or dependency failure that caused the fault. This replaces the traditional, error-prone process of “grepping logs” across multiple servers with formal causality tracing, enabling operators to pinpoint root causes with mathematical certainty.

13.3 Formal Invariant Verification

Integrating the Design Rules and Assertions from the W2P strategy, the Orchestrator natively supports *Assertion Steps*. Before a mutating command executes, the engine verifies a required mathematical network invariant (e.g., “BGP adjacency matrix must be fully connected”). Post-mutation, the invariant is re-verified.

This mechanism is analogous to Dijkstra’s self-stabilizing systems [7], ensuring that the system only transitions between provably safe states. If an assertion fails, the target is automatically quarantined. This shifts the definition of success from “the command did not error” to “the network maintained structural integrity.”

13.4 Predictive Blast Radius Simulation

By combining the simulation mode (*dry_run*) with the *netnsim* adapter, the Orchestrator performs a full pre-flight simulation before any physical hardware is touched. It computes the resulting state changes against a simulated network topology and outputs a formal “Blast Radius” report. This report mathematically quantifies which segments of the topology will be impacted (e.g., specific routing domains or VLANs). Making risk assessment an automated calculation rather than a human estimation drastically reduces the likelihood of catastrophic outages during maintenance windows.

14 Formal State Attestation and Cryptographic Compliance

In high-stakes operational environments, compliance is traditionally a manual, fallible process. Orchestrator v2.0 elevates execution from a mere change operation to a mathematical authority by introducing *Formal State Attestation*.

The engine transforms the linear event log into a cryptographically verifiable record of truth. Every emitted event, SMT proof result, and redacted artifact hash is rolled into a *Merkle Tree* [8], providing an immutable chain of custody for the entire run.

Furthermore, drawing on Zero-Knowledge Proof (ZKP) theory [9], the Orchestrator generates a compact cryptographic proof that the network maintained all formal invariants during the maintenance window. An external auditor can verify this proof to guarantee operational correctness without ever accessing sensitive router configurations or plaintext credentials. By anchoring these proofs to hardware-level identifiers (e.g., TPM-signed serial numbers), the system provides a non-repudiable ledger of network intent and execution.

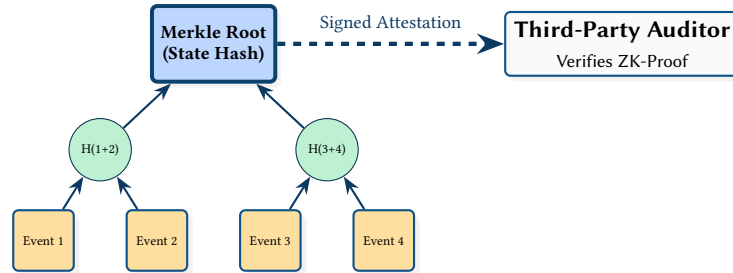


Figure 17. Formal State Attestation: Events are rolled into a Merkle Tree to provide a cryptographically verifiable audit trail.

15 Shadow-Fabric Pre-Commit (Ephemeral Digital Twins)

To bridge the gap between simulation and physical reality, the Orchestrator implements *Shadow-Fabric Pre-Commit*. This feature utilizes the concept of the *Digital Twin* [10] to provide a high-fidelity "Shadow DOM" for physical network infrastructure.

For every mutating step in the execution DAG, the engine JIT-spins up a containerized, protocol-accurate twin of the target hardware. The mutation is first applied to this ephemeral twin, where the Orchestrator executes its full suite of *Formal Invariant Verifications*. Only if the shadow mutation achieves the desired state transition does the engine proceed to push the configuration to the physical data-plane. This nested execution model ensures that even non-modeled emergent behaviors are captured in a safe environment before impacting live traffic.

16 Implementation Walkthrough (Repository Mapping)

This section maps the conceptual components to concrete modules, moving from the foundational models into the executable architecture.

Module	Responsibility
src/orchestrator/api/*	FastAPI app, routers for workflows, runs, and SSE event streams
src/orchestrator/models/*	Pydantic models (Workflow, Target, Run, ArtifactMetadata, OperationalTopology)
src/orchestrator/engine/state.py	RunStateRepository protocol + SQLite implementation (events, artifacts)
src/orchestrator/engine/supervisor.py	Run fan-out, dependency sorting (DAG), and daemon lifecycle
src/orchestrator/engine/adapters/*	Target adapters (netsim, device_interaction, mock)
src/orchestrator/engine/safety.py	CommandValidator and allow/deny lists
src/orchestrator/engine/redactor.py	Automated zero-knowledge scrubbing from artifacts and logs
src/orchestrator/cli/*	Native CLI (Typer) for executing runs and streaming logs
src/orchestrator/toolchain/*	Extracted global toolchain orchestrator

Table 1. Code-to-architecture mapping for the v1 Orchestrator implementation.

Design Decision 4: Protocol-driven State

The supervisor depends on a RunStateRepository protocol. This keeps the execution engine decoupled from a specific persistence mechanism (SQLite in v0.1.0, but replaceable for v2.0 CRDT backends).

17 v2.0 Architecture: Stochastic Workflows and Distributed Control

The v1.0.0 implementation realizes the core control-plane contracts and provides a complete device-first execution engine (including real/simulated adapters, durable event logs, and automated redaction). The next phase of development transitions the Orchestrator from a deterministic runbook engine into a robust, distributed control system capable of handling the inherent entropy of physical networks.

Drawing inspiration from advanced data orchestration pipelines and distributed systems theory, the v2.0 roadmap focuses on stochastic simulation, topological awareness, and decentralized execution.

17.1 Stochastic DAG Evaluation (Monte Carlo Stress Testing)

Determinism does not mean ignoring real-world entropy; it means rigorously quantifying an execution graph’s resilience to anticipated network noise. v2.0 will introduce *Probabilistic Fault Simulation*, leveraging Monte Carlo methods [11].

Before execution, operators can annotate the workflow DSL with failure probabilities for specific edges or adapters. The engine performs thousands of simulated runs over the execution DAG—injecting modeled network entropy such as packet loss, transient authentication timeouts, or latency spikes into the `netsim` environment. The output is a formal confidence interval (e.g., “This deployment has a 99.7% probability of reaching a terminal invariant-safe state”), allowing teams to mathematically stress-test workflows prior to production rollout.

17.2 Graph-Theoretic Concurrency and Blast-Radius Containment

Currently, concurrency is bounded by flat token budgets. v2.0 elevates this to *Topology-Aware Scheduling*. The scheduler ingests the Operational Topology as a directed graph. Using graph-coloring algorithms and max-flow min-cut theorems, the engine calculates mathematically safe concurrency lanes. It guarantees that simultaneous mutations never occur on redundant routing paths or adjacent spine switches, ensuring the network always maintains a viable forwarding path during distributed execution.

17.3 Algebraic State Reversion (Automated Compensating Transactions)

When a multi-device workflow fails halfway through, standard runbooks leave the network in a fractured, inconsistent state. v2.0 addresses this by mapping adapter actions to *Inverse Functors*. Because the Orchestrator maintains an immutable SQLite event log of exactly what succeeded, it can dynamically synthesize a “Compensating DAG.” It generates and executes the exact inverse operations required to gracefully roll the entire network fabric back to the last known-good topological state, effectively providing transactional rollbacks for physical infrastructure.

17.4 Semantic Convergence Bounding

Sending a command is instantaneous, but routing protocols take time to converge. Standard orchestrators rely on arbitrary `sleep()` intervals that lead to race conditions. v2.0 introduces *Convergence Barriers* to the workflow DSL. Instead of sleeping, the execution graph actively yields and subscribes to the network’s telemetry plane. The DAG only unblocks when a mathematical invariant is met (e.g., “BGP route-reflector table size first derivative approaches zero”). This ensures that distributed execution accurately tracks true protocol convergence.

17.5 Decentralized Edge Federation via CRDTs

Managing a global network from a centralized orchestrator introduces latency and partition risks. v2.0 supports a headless, federated execution mode utilizing Conflict-Free Replicated Data Types (CRDTs) [12] over local SQLite event logs.

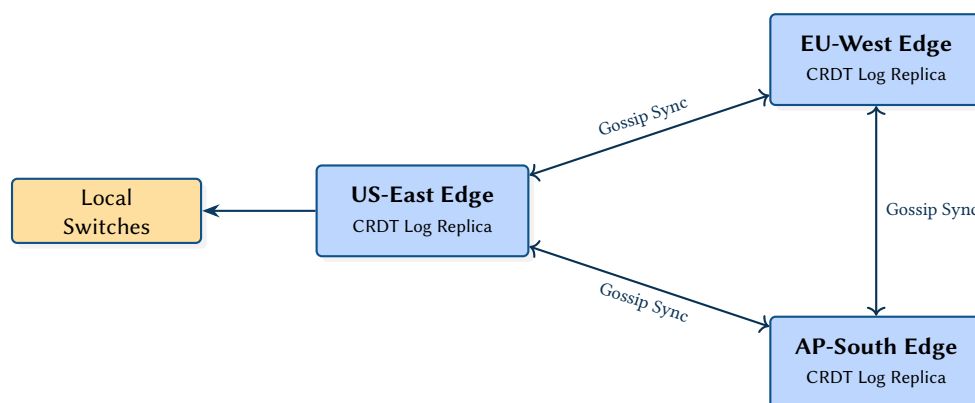


Figure 18. Decentralized Federation: Edge orchestrators execute local DAG segments and gossip state via CRDTs, eliminating the single point of failure.

Multiple Orchestrator engines can be deployed directly at the network edge. They independently execute local sub-graphs of the W2P Blueprint. As network partitions heal, these edge instances gossip their local event streams to synchronize into a globally consistent consensus state. By integrating principles from Byzantine Fault Tolerance (BFT) [13], the federated system ensures that isolated faults or compromised edge nodes cannot corrupt the global intent.

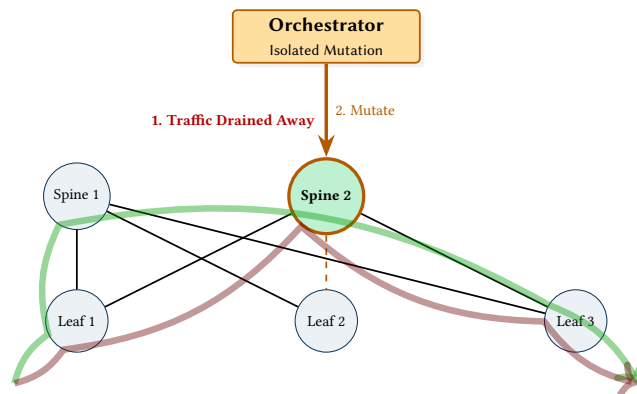
17.6 Dynamic Topological Sandboxing (Execution-Driven Traffic Engineering)

The absolute pinnacle of Orchestrator’s v2.0 capabilities is *Dynamic Topological Sandboxing*. In traditional software, CI/CD pipelines isolate workloads behind load balancers. In physical networking, the network fabric is the load balancer, making maintenance windows on live data-planes inherently high-risk.

By combining the engine’s topological awareness with advanced traffic engineering (TE) principles [14], the Orchestrator autonomously creates ephemeral, zero-impact maintenance windows. When a workflow targets a critical core node, the engine executes a four-stage sandbox lifecycle:

1. **Pre-Flight Drain:** The engine dynamically synthesizes and executes a transient execution graph that manipulates routing metrics (e.g., BGP local-preference or OSPF costs) to shift all live data-plane traffic away from the target node.
2. **Telemetry Yield:** Utilizing the *Convergence Barriers* defined earlier, the Orchestrator actively monitors the hardware telemetry plane until it is mathematically proven that the node is passing zero transit packets.
3. **Isolated Execution:** The target node is now “dark” to the data-plane but fully accessible to the Orchestrator’s control-plane. The dangerously mutating workflow executes in complete isolation, with zero risk of packet loss.
4. **Post-Flight Reintroduction:** Once *Formal Invariant Verification* passes, the engine restores the original routing metrics, seamlessly reintroducing the validated node to the live fabric.

Drawing on the intellectual history of programmable networks and SDN control loops [15], this feature elevates the Orchestrator from a command runner to a system that actively bends physical reality to guarantee safe execution conditions.



The Orchestrator manipulates the control-plane to create a transient data-plane sandbox, reintroducing the node only after all assertions pass.

Figure 19. Dynamic Topological Sandboxing: The engine shifts traffic away from the mutation target to create a zero-impact maintenance window.

17.7 Symbolic Execution and Formal Verification (SMT Theorem Proving)

To achieve absolute mathematical rigor in network state management, Orchestrator v2.0 introduces *Symbolic Execution of Infrastructure*. While empirical simulation (netsim) can prove the *presence* of bugs, symbolic execution is required to prove their *absence*.

Drawing on foundational techniques like Header Space Analysis [16] and real-time invariant verification [17], the engine treats the network not as a collection of configuration targets, but as a mathematical theorem to be proved. Before an execution DAG is scheduled, the Orchestrator compiles the W2P Blueprint and the current topological state into a set of first-order logic formulas.

These formulas are fed into a high-performance SMT solver (e.g., Z3). The solver explores the entire state space of the network—mathematically evaluating the impact of the proposed changes across all possible packet headers and all possible external route injections. The engine only proceeds if the solver proves that the post-deployment state will remain free of routing loops, blackholes, or security policy violations. This

shift from testing to formal proof provides the ultimate guarantee of structural integrity for mission-critical infrastructure.

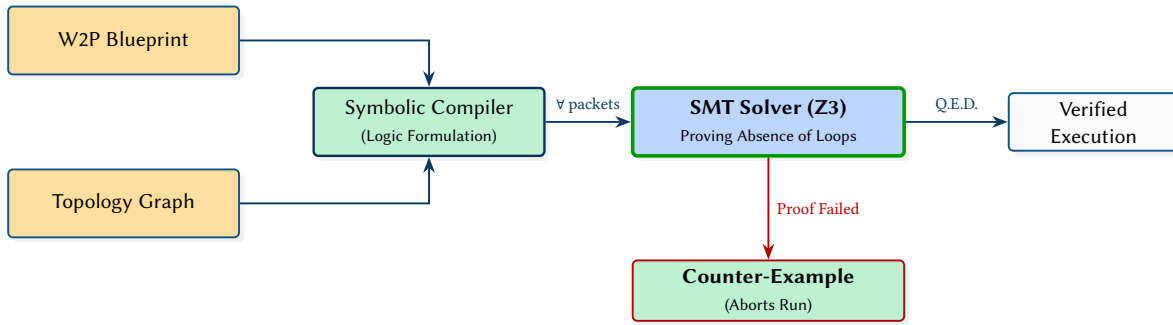


Figure 20. Symbolic Execution: The Orchestrator uses SMT solvers to prove network invariants before a single command is dispatched.

: Preserve Event Semantics

Any expansion of execution capabilities (stochastic simulation, automated reversion, CRDT federation, topological sandboxing, or symbolic proof) must preserve the stable event correlation identifiers and append-only log semantics established in v1.

References

- [1] *Orchestrator project definition (local)*, File: .planning/PROJECT.md in /Users/simonknight/dev/orchestrator, Scope, core value, constraints, and key decisions., 2026.
- [2] *Orchestrator requirements (local)*, File: .planning/REQUIREMENTS.md in /Users/simonknight/dev/orchestrator, v1 requirements and traceability to phases., 2026.
- [3] *Orchestrator roadmap (local)*, File: .planning/ROADMAP.md in /Users/simonknight/dev/orchestrator, Phase sequencing and success criteria for v1., 2026.
- [4] *Server-sent events*, WHATWG HTML Living Standard, EventSource interface and SSE event stream format., 2026.
- [5] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, “Intent-based networking - concepts and definitions,” IETF, RFC 9315, 2022.
- [6] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [7] E. W. Dijkstra, “Self-stabilizing systems in spite of distributed control,” *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, 1974.
- [8] R. C. Merkle, “A digital signature based on conventional encryption,” in *Advances in Cryptology — CRYPTO ’87*, 1988, pp. 369–378.
- [9] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.
- [10] M. Grieves and J. Vickers, “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” *Transdisciplinary Perspectives on Complex Systems*, pp. 85–113, 2017.
- [11] N. Metropolis and S. Ulam, “The monte carlo method,” *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949.
- [12] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” *Stabilization, Safety, and Security of Distributed Systems*, pp. 386–400, 2011.
- [13] M. Castro and B. Liskov, “Practical byzantine fault tolerance,” *OSDI*, vol. 99, pp. 173–186, 1999.
- [14] D. Awduche, A. Chiu, A. Elwalid, I. Widjaja, and X. Xiao, “Overview and principles of internet traffic engineering,” *RFC 3272*, 2002.
- [15] N. Feamster, J. Rexford, and E. Zegura, “The road to sdn: An intellectual history of programmable networks,” *ACM SIGCOMM Computer Communication Review*, vol. 44, pp. 87–107, 2014.
- [16] P. Kazemian, G. Varghese, and N. McKeown, “Header space analysis: Static checking for networks,” in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012, pp. 113–126.
- [17] A. Khurshid, X. Zou, W. Wen, and M. Caesar, “Veriflow: Verifying network-wide invariants in real time,” pp. 15–27, 2013.

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial technical report for v0.1.0 implementation
1.0.0	March 2026	v1 MVP complete: adapters, artifacts, observability, safety, and toolchain extraction